

Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming

Hussein Mozannar
mozannar@mit.edu
Massachusetts Institute of Technology
Boston, USA

Adam Fourney
adamfo@microsoft.com
Microsoft Research
Redmond, USA

Gagan Bansal
gaganbansal@microsoft.com
Microsoft Research
Redmond, USA

Eric Horvitz
horvitz@microsoft.com
Microsoft Research
Redmond, USA

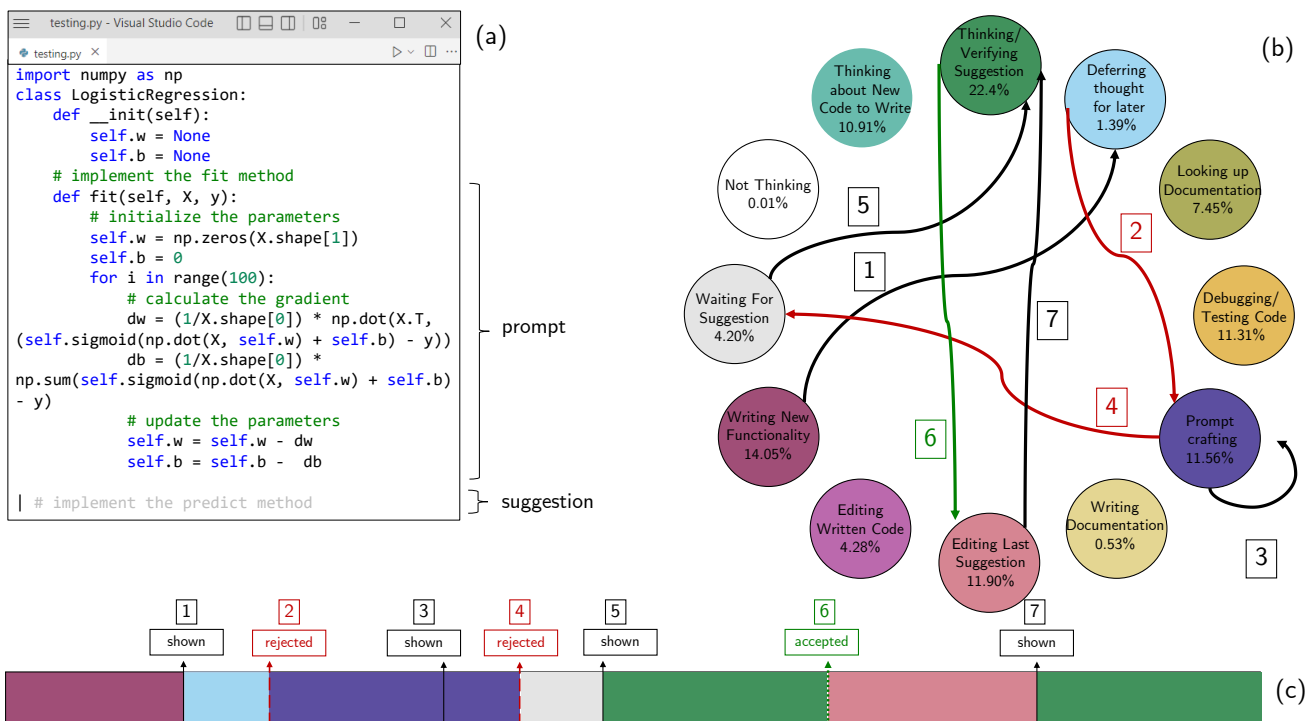


Figure 1: Profiling a coding session with the CodeRec User Programming States (CUPS). In (a) we show the operating mode of CodeRec inside Visual Studio Code. In (b) we show the CUPS taxonomy used to describe CodeRec related programmer activities. A coding session can be summarized as a timeline in (c) where the programmer transitions between states.

ABSTRACT

Code-recommendation systems, such as Copilot and CodeWhisperer, have the potential to improve programmer productivity by suggesting and auto-completing code. However, to fully realize their potential, we must understand how programmers interact with these systems and identify ways to improve that interaction. To seek insights about human-AI collaboration with code recommendations systems, we studied GitHub Copilot, a code-recommendation system used by millions of programmers daily. We developed CUPS, a taxonomy of common programmer activities when interacting



This work is licensed under a Creative Commons Attribution International 4.0 License.

CHI '24, May 11–16, 2024, Honolulu, HI, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0330-0/24/05
<https://doi.org/10.1145/3613904.3641936>

with Copilot. Our study of 21 programmers, who completed coding tasks and retrospectively labeled their sessions with CUPS, showed that CUPS can help us understand how programmers interact with code-recommendation systems, revealing inefficiencies and time costs. Our insights reveal how programmers interact with Copilot and motivate new interface designs and metrics.

CCS CONCEPTS

• **Human-centered computing** → **User models; User studies;** • **Software and its engineering** → **Automatic programming.**

KEYWORDS

AI-assisted Programming, Copilot, User State Model

ACM Reference Format:

Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3613904.3641936>

1 INTRODUCTION

Programming-assistance systems based on the adaptation of large language models (LLMs) to code recommendations have been recently introduced to the public. Popular systems, including Copilot [14], CodeWhisperer [1], and AlphaCode[21], signal a potential shift in how software is developed. Though there are differences in specific interaction mechanisms, the programming-assistance systems generally extend existing IDE code completion mechanisms (e.g., IntelliSense¹) by producing suggestions using neural models trained on billions of lines of code [8]. The LLM-based completion models can suggest sentence-level completions to entire functions and classes in a wide array of programming languages. These large neural models are deployed with the goal of accelerating the efforts of software engineers, reducing their workloads, and improving their productivity.

Early assessments suggest that programmers do feel more productive when assisted by the code recommendation models [40] and that they prefer these systems to earlier code completion engines [34]. In fact, a recent study from GitHub, found that Copilot could potentially reduce task completion time by a factor of two [28]. While these studies help us understand the benefits of code-recommendation systems, they do not allow us to identify avenues to improve and understand the nature of interaction with these systems.

In particular, the neural models introduce new tasks into a developer's workflow, such as writing AI prompts [17] and verifying AI suggestions [34], which can be lengthy. Existing interaction metrics, such as suggestion acceptance rates, time to accept (i.e., the time a suggestion remains onscreen), and reduction of tokens typed, tell only part of this interaction story. For example, when suggestions are presented in monochrome popups (Figure 1), programmers may choose to accept them into their codebases so that they can be read with code highlighting enabled. Likewise, when models suggest only one line of code at a time, programmers may accept sequences

before evaluating them together as a unit. In both scenarios, considerable work verifying and editing suggestions occurs *after* the programmer has accepted the recommended code. Prior interaction metrics also largely miss user effort invested in devising and refining prompts used to query the models. When code completion tools are evaluated using coarser task-level metrics such as task completion time [20], we begin to see signals of the benefits of AI-driven code completion but lack sufficient detail to understand the nature of these gains, as well as possible remaining inefficiencies. We argue that an ideal approach would be sufficiently low level to support interaction profiling while sufficiently high level to capture meaningful programmer activities.

Given the nascent nature of these systems, numerous questions exist regarding the behavior of their users:

- What activities do users undertake in anticipation for, or to trigger a suggestion?
- What mental processes occur while the suggestions are onscreen, and, do people double-check suggestions before or after acceptance?
- How *costly* for users are these various new tasks, and which take the most time?

To answer these and related questions in a systematic manner, we apply a mixed-methods approach to analyze interactions with a popular code suggestion model, GitHub Copilot² which has more than a million users. To emphasize that our analysis is not restricted to the specifics of Copilot, we use the term CodeRec to refer to any instance of code suggestion models, including Copilot. Through small-scale pilot studies and our first-hand experience using Copilot for development, we develop a novel taxonomy of common states of a programmer when interacting with CodeRec models (such as Copilot), which we refer to as CodeRec User Programming States (CUPS). The CUPS taxonomy serves as the main tool to answer our research questions.

Given the initial taxonomy, we conducted a user study with 21 developers who were asked to retrospectively review videos of their coding sessions and explicitly label their intents and actions using this model, with an option to add new states if necessary. The study participants labeled a total of 3137 coding segments and interacted with 1096 suggestions. The study confirmed that the taxonomy was sufficiently expressive, and we further learned transition weights and state dwell times—something we could not do without this experimental setting. Together, these data can be assembled into various instruments, such as the CUPS diagram (Figure 1), to facilitate profiling interactions and identify inefficiencies. Moreover, we show that such analysis nearly doubles our estimates for how much developer time can be attributed to interactions with code suggestion systems, as compared with existing metrics. We believe that identifying the current CUPS state during a programming session can help serve programmer needs. This can be accomplished using custom keyboard macros or automated prediction of CUPS states, as discussed in our future work section and the Appendix. Overall, we leverage the CUPS diagram to identify some opportunities to address inefficiencies in the current version of Copilot.

In sum, our main contributions are the following:

¹<https://code.visualstudio.com/docs/editor/intellisense>

²<https://github.com/features/copilot>

- A novel taxonomy of common activities of programmers (called CUPS) when interacting with code recommendation systems (Section 4)
- A dataset of coding sessions annotated with user actions, CUPS, and video recordings of programmers coding with Copilot (Section 5).
- Analysis of which CUPS states programmers spend their time in when completing coding tasks (Subsection 6.1).
- An instrument to analyze programmer behavior (and patterns in behavior) based on a finite-state machine on CUPS states (Subsection 6.2).
- An adjustment formula to properly account for how much time do programmers spend verifying CodeRec suggestions (Subsection 6.4) inspired by the CUPS state of deferring thought (Subsection 6.3).

The remainder of this paper is structured as follows: We first review related work on AI-assisted programming (Section 2) and formally describe Copilot, along with a high-level overview of programmer-CodeRec interaction (Section 3). To further understand this interaction, we define our model of CodeRec User Programming States (CUPS) (Section 3) and then describe a user study designed to collect programmer annotations of their states (Section 5). We use the collected data to analyze the interactions using CUPS diagram revealing new insights into programmer behavior (Section 6). We then discuss limitations and future work and conclude in (Section 7).

2 BACKGROUND AND RELATED WORK

Large language models based on the Transformer network [36], such as GPT-3 [6], have found numerous applications in natural language processing. Codex [8], a GPT model trained on 54 million GitHub repositories, demonstrates that LLMs can very effectively solve various programming tasks. Specifically, Codex was initially tested on the HumanEval dataset containing 164 programming problems, where it is asked to write the function body from a docstring [8] and achieves 37.7% accuracy with a single generation. Various metrics and datasets have been proposed to measure the performance of code generation models [9, 11, 16, 21]. However, in each case, these metrics test how well the model can complete code in an offline setting without developer input rather than evaluating how well such recommendations assist programmers in situ. This issue has also been noted in earlier work on non-LLM based code completion models where performance on completion benchmarks overestimates the model's utility to developers [15]. Importantly, however, these results may not hold to LLM-based approaches, which are radically different [30].

One straightforward approach to understanding the utility of neural code completion services, including their propensity to deliver incomplete or imperfect suggestions, is to simply ask developers. To this end, Weisz et al. interviewed developers and found that they did not require a perfect recommendation model for the model to be useful [38]. Likewise, Ziegler et al. surveyed over 2,000 Copilot users [40] and asked about perceived productivity gains using a survey instrument based on the SPACE framework [13] – we incorporate the same survey design for our own study. They found both that developers felt more productive using Copilot and

that these self-reported perceptions were reasonably correlated with suggestion acceptance rates. Liang et al. [22] administered a survey to 410 programmers who use various AI programming assistants, including Copilot, and highlighted why the programmers use the AI assistants and numerous usability issues. Similarly, Prather et al. [29] surveyed how introductory programming students utilize Copilot.

While these self-reported measures of utility and preference are promising, we would expect gains to be reflected in objective metrics of productivity. Indeed, one ideal method would be to conduct randomized control trials where one set of participants writes code with a recommendation engine while another set codes without it. GitHub performed such an experiment where 95 participants were split into two groups and asked to write a web server. The study concluded by finding that task completion was reduced by 55.8% in the Copilot condition [28]. Likewise, a study by Google showed that an internal CodeRec model had a 6% reduction in 'coding iteration time' [33]. On the other hand, Vaithilingam et al. [34] showed in a study of 24 participants showed no significant improvement in task completion time – yet participants stated a clear preference for Copilot. An interesting comparison to Copilot is Human-Human pair programming, which Wu et al. [39] details.

A significant amount of work has tried to understand the behavior of programmers [4, 5, 23, 31] using structured user studies under the name of "psychology of programming." This line of work tries to understand the effect of programming tools on the time to solve a task or ease of writing code and how programmers read and write code. Researchers often use telemetry with detailed logging on keystrokes [19, 37] to understand behavior. Moreover, eye-tracking is also used to understand how programmers read code [24, 27]. Our research uses raw telemetry alongside user-labeled states to understand behavior; future research could also utilize eye-tracking and raw video to get deeper insights into behavior.

This wide dispersion of results raises interesting questions about the nature of the utility afforded by neural code completion engines: how, and when, are such systems most helpful; and conversely, when do they add additional overhead? **This is the central question to our work.** The related work closest to answering this question is that of Barke et al. [3], who showed that interaction with Copilot falls into two broad categories: the programmer is either in "acceleration mode" where they know what they want to do, and Copilot serves to make them faster; or they are in "exploration mode", where they are unsure what code to write and Copilot helps them explore. The taxonomy we present in this paper, CUPS, enriches this further with granular labels for programmers' intents. Moreover, the data collected in this work was labeled by the participants themselves rather than by the researchers interpreting their actions, allowing for more faithful intent and activity labeling and the data collected in our study can also be used to build predictive models as in [32]. The next section describes the Copilot system formally and describes the data collected when interacting with Copilot.

3 COPILOT SYSTEM DESCRIPTION

To better understand how code recommendation systems influence the effort of programming, we focus on GitHub Copilot, a popular

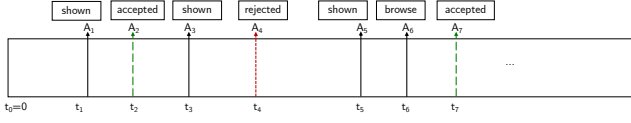


Figure 2: Schematic of interaction telemetry with Copilot as a timeline. For a given coding session, the telemetry contains a sequence of timestamps and actions with associated prompt and suggestion features (not shown).

and representative example of this class of tools. Copilot³ is based on a Large Language Model (LLM) and assists programmers inside an IDE by recommending code suggestions any time the programmer pauses their typing. Figure 1 shows an example of Copilot recommending a code snippet as an inline, monochrome popup, which the programmer can accept using a keyboard shortcut (e.g., <tab>).

To serve suggestions, Copilot uses a portion of the code written so far as a *prompt*, P , which it passes to the underlying LLM. The model then generates a suggestion, S , which it deems to be a likely completion. In this regime, programmers can *engineer* the prompt to generate better suggestions by carefully authoring natural language comments in the code such as “# split the data into train and test sets.” In response to a Copilot suggestion, the programmer can then take one of several actions A , where $A \in \{\text{browse, accept, reject}\}$. The latter of these actions, *reject*, is triggered implicitly by continuing to type something that differs from the suggestion or by pressing the escape key. The browse action enables the programmer to change the suggestion shown with a keyboard shortcut from a set of at most three suggestions. Copilot logs aspects of the interactions via *telemetry*. We leverage this telemetry in the studies described in this paper. Specifically, whenever a suggestion is shown, accepted, rejected, or browsed, we record a tuple to the telemetry database, (t_i, A_i, P_i, S_i) , where t_i represents the within-session timestamp of the i^{th} event ($t_0 = 0$), A_i details the action taken (augmented to include ‘shown’), and P_i and S_i capture features of the prompt and suggestion, respectively. Figure 2 displays telemetry of a coding session, and Figure 1a shows Copilot implemented as a VSCode plugin. We have the ability to capture telemetry for any programmer interacting with Copilot; this is used to collect data for a user study in section 5.

3.1 Influences of CodeRec on Programmer’s Activities

Despite the limited changes that Copilot introduces to an IDE’s repertoire of actions, LLM-based code suggestions can significantly influence how programmers author code. Specifically, Copilot leverages LLMs to stochastically *generate* novel code to fit the arbitrary current context. As such, the suggestions may contain errors (and can appear to be unpredictable) and require that programmers double-check and edit them for correctness. Furthermore, programmers may have to refine the prompts to get the best suggestions. These novel activities associated with the AI system introduce new efforts and potential disruptions to the flow of programming. **We**

use time as a proxy to study the new costs of interaction introduced by the AI system. We recognize that this approach is incomplete: the costs associated with solving programming tasks are multi-dimensional, and it can be challenging to assign a single real-valued number to cover all facets of the task [12]. Nevertheless, we argue that, like accuracy, efficiency-capturing measures of time are an important dimension of the cost that is relevant to most programmers.

3.2 Programmer Activities in Telemetry Segments

Copilot’s telemetry captures only instantaneous user actions (e.g., accept, reject, browser), as well as the suggestion display event. By themselves, these entries do not reveal such programmer’s activities as double-checking and prompt engineering, as such activities happen *between* two consecutive instantaneous events. **We argue that the regions between events, which we refer to as *telemetry segments*, contain important user intentions and activities unique to programmer-CodeRec interaction**, which we need to understand in order to answer how Copilot affects programmers—and where and when Copilot suggestions are useful to programmers.

Building on this idea, telemetry segments can be split into two groups (Figure 2). The first group includes segments that start with a suggestion shown event and end with an action (accept, reject, or browse). Here, the programmer is paused and has yet to take action. We refer to this as ‘User Before Action’. The second group includes segments that start with an action event and end with a display event. During this period, the programmer can be either typing or paused; hence we denote it as ‘User Typing or Paused’. These two groups form the foundation of a deeper taxonomy of programmers’ activities, which we will further develop in the next section.

4 A TAXONOMY FOR UNDERSTANDING PROGRAMMER-CODEREC INTERACTION: CUPS

4.1 Creating the Taxonomy

Our objective is to create an extensive, but not complete, taxonomy of programmer activities when interacting with CodeRec that enables a useful study of the interaction. To refine the taxonomy of programmers’ activities, we developed a labeling tool and populated it with an initial set of activities based on our own experiences from extensive interactions with Copilot (Figure 4). The tool enables users to watch a recently captured screen recording of them solving a programming task with Copilot’s assistance and to *retrospectively* annotate each telemetry segment with an activity label. We use this tool to first refine our taxonomy with a small pilot study (described below) and then to collect data in Section 5.

The labeling tool (Figure 4) contains three main sections: a) A navigation panel on the left, which displays and allows navigating between telemetry segments and highlights the current segment being labeled in blue. The mouse or arrow keys are used to navigate between segments. b) A video player on the right, which plays the corresponding video segments in a loop. The participant can watch the video segments any number of times. c) Buttons on the bottom

³The version of Copilot that this manuscript refers to is Copilot as of August 2022.

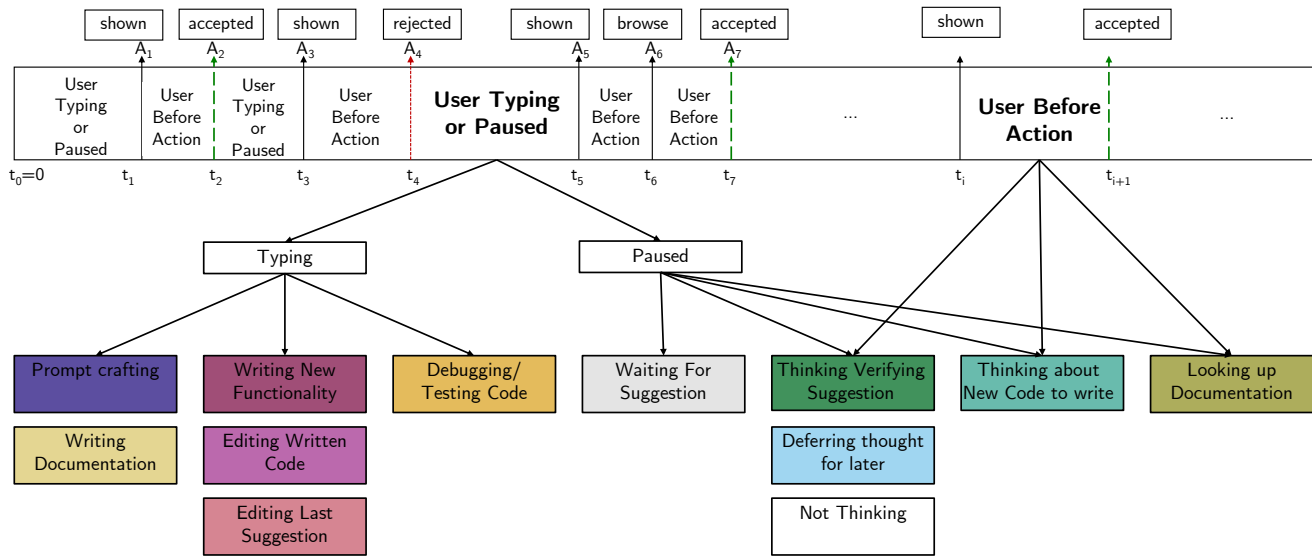


Figure 3: Taxonomy of programmer's activities when interacting with CodeRec- CUPS.

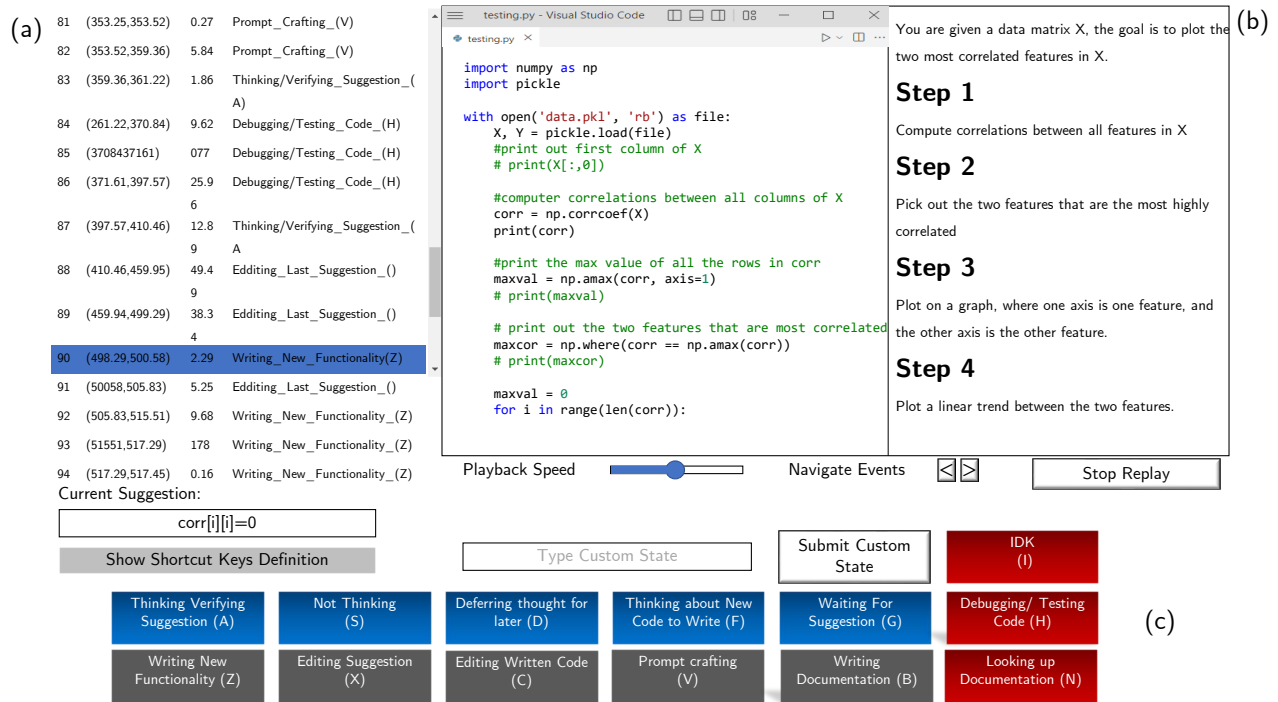


Figure 4: Screenshot of retrospective labeling tool for coding sessions. Left: Navigation panel for telemetry segments. Right: Video player for reviewing video of a coding session. Bottom: Buttons and text box for labeling states.

corresponding to the CUPS taxonomy, along with an “IDK” button

and a free-form text box to write custom state labels. Buttons also have associated keyboard bindings for easy annotation.

Table 1: Description of each state in CodeRec User Programming States (CUPS).

State	Description
Thinking/Verifying Suggestion	Actively thinking about and verifying a shown or accepted suggestion
Not Thinking	Not thinking about suggestion or code, programmer away from keyboard
Deferring Thought For Later	Programmer accepts suggestion without completely verifying it, but plans to verify it after
Thinking About New Code To Write	Thinking about what code or functionality to implement and write
Waiting For Suggestion	Waiting for CodeRec suggestion to be shown
Writing New Code	Writing code that implements new functionality
Editing Last Suggestion	Editing the last accepted suggestion
Editing (Personally) Written Code	Editing code written by a programmer that is not a CodeRec suggestion for the purpose of fixing existing functionality
Prompt Crafting	Writing prompt in the form of comment or code to obtain desired CodeRec suggestion
Writing Documentation	Writing comments or docstring for purpose of documentation
Debugging/Testing Code	Running or debugging code to check functionality may include writing tests or debugging statements
Looking up Documentation	Checking an external source for the purpose of understanding code functionality (e.g. Stack Overflow)
Accepted	Accepted a CodeRec suggestion
Rejected	Rejected a CodeRec suggestion

To label a particular video segment, we asked participants to consider the hierarchical structure of CUPS in Figure 3. The hierarchical structure first distinguishes segments by whether a typing segment occurred in that segment and then decides based on the typing or non-typing states. For example, in a segment where a participant was initially double-checking a suggestion and then wrote new code to accomplish a task, the appropriate label would be "Writing New Functionality" as the user eventually typed in the segment. In cases where there are two states that are appropriate and fall under the same hierarchy, e.g., if the participant double-checked a suggestion and then looked up documentation, they were asked to pick the state in which they spent the majority of the time. These issues arise because we collect a single state for each telemetry segment.

Pilot. Through a series of pilots involving the authors of the paper, as well as three other participants drawn from our organization, we iteratively applied the tool to our own coding sessions and to the user study tasks described in section 5. We then expanded and refined the taxonomy by incorporating any "custom state" (using the text field) written by the pilot participants. The states 'Debugging/Testing Code', 'Looking up Documentation', and

'Writing Documentation' were added through the pilots. By the last pilot participant, the code book was stable and saturated as they did not write a state that was not yet covered. We observed in our study that the custom text field was rarely used. We describe the resultant taxonomy in the sections below.

4.2 Taxonomy of Telemetry Segments

Figure 3 shows the finalized taxonomy of programmer activities for individual telemetry segments with Copilot. As noted earlier, the taxonomy is rooted in two segment types: 'User Typing or Paused', and 'User Before Action'. We first detail the 'User Typing or Paused' segments, which precede shown events (Figure 2) and are distinguished by the fact that no suggestions are displayed during this time. As the name implies, users can find themselves in this state if they are either actively 'Typing'⁴, or have 'paused' but have not yet been presented with a suggestion. In cases where the programmer is actively typing, they could be completing any of a number of tasks such as: 'writing new functionality,' 'editing existing code,' 'editing prior (CodeRec) suggestions,' 'debugging code,' or authoring natural language comments, including both documentation and prompts directed at CodeRec (i.e., 'prompt crafting'). When the user pauses, they may simply be "waiting for a suggestion" or can be in any number of states common to 'User Before Action' segments.

In every 'User Before Action' segment, CodeRec is displaying a suggestion, and the programmer is paused and not typing. They could be reflecting and verifying that suggestion, or they may not be paying attention to the suggestion and thinking about other code to write instead. The programmer can also *defer* their efforts on the suggestion for a later time period by accepting it immediately, then pausing to review the code at a later time. This can occur, for example, because the programmer desires syntax highlighting rather than grey text or because the suggestion is incomplete, and the programmer wants to allow Copilot to complete its implementation before evaluating the code as a cohesive unit. The latter situation tends to arise when Copilot displays code suggestions line by line (e.g., Figure 7).

The leaf nodes of the finalized taxonomy represent 12 distinct states that programmers can find themselves in. These states are illustrated in Figure 3 and are further described in Table 1. While the states are meant to be distinct, siblings may share many traits. For example, "Writing New Functionality" and "Editing Written Code" are conceptually very similar. This taxonomy also bears resemblance to the keystroke level model in that it assigns a time cost to mental processes as well as typing [7, 18]. As evidenced by the user study—which we describe in the next section—these 12 states provide a language that is both *general* enough to capture most activities (at this level of abstraction), and *specific* enough to meaningfully capture activities unique to LLM-based code suggestion systems.

5 CUPS DATA COLLECTION STUDY

To study CodeRec-programmer interaction in terms of CodeRec User Programming States, we designed a user study where programmers perform a coding task, then review and label videos of their coding session using the telemetry segment-labeling tool described

⁴Active typing allows for brief pauses between keystrokes.

earlier. We describe the procedure, the participants, and the results in the sections that follow.

5.1 Procedure

We conducted the study over a video call and asked participants to use a remote desktop application to access a virtual machine (VM). Upon connecting, participants were greeted with the study environment consisting of Windows 10, together with Visual Studio Code (VS Code) augmented with the Copilot plugin.

Participants were then presented with a programming task drawn randomly from a set of eight pre-selected tasks (Table 2). If the participant was unfamiliar with the task content, we offered them a different random task. The task set was designed during the pilot phase so that individual tasks fit within a 20-minute block and so that, together, the collection of tasks surfaces a sufficient diversity of programmer activities. It is crucial that the task is of reasonable duration so that participants are able to remember all their activities since they will be required to label their session immediately afterward. Since the CUPS taxonomy includes states of thought, participants must label their session immediately after coding, and each study took approximately 60 minutes in total. To further improve diversity, task instructions were presented to participants as images to encourage participants to author their own Copilot prompts rather than copying and pasting from the problem description. The full set of tasks and instructions is provided as an Appendix.

Upon completing the task (or reaching the 20-minute mark), we loaded the participant’s screen recording and telemetry into the labeling tool (previously detailed in Section 4.1). The researcher then briefly demonstrated the correct operation of the tool and explained the CUPS taxonomy. Participants were then asked to annotate their coding session with CUPS labels. Self-labeling allows us to easily scale such a study and enables more accurate labels for each participant, but may cause inconsistent labeling across participants. Critically, this labeling occurred within minutes of completing the programming task so as to ensure accurate recall. We do not include a baseline condition where participants perform the coding task without Copilot, as this work focuses on understanding and modeling the interaction with the current version of Copilot.

Finally, participants completed a post-study questionnaire about their experience mimicking the one in [40]. The entire experiment was designed to last 60 minutes. The study was approved by our institutional review board (IRB), and participants received a \$50.00 gift card as remuneration for their participation.

5.2 Participants

To recruit participants, we posted invitations to developer-focused email distribution lists within our large organization. We recruited 21 participants with varying degrees of experience using Copilot: 7 used Copilot more than a few times a week, 3 used it once a month or less, and 11 had never used it before. For participants who had never used it before, the experimenter gave a short oral tutorial on Copilot explaining how it can be invoked and how to accept suggestions. Participants’ roles in the organization ranged from software engineers (with different levels of seniority) to researchers

and graduate student interns. In terms of programming expertise, only 6 participants had less than 2 years of professional programming experience (i.e., excluding years spent learning to program), 5 had between 3 to 5 years, 7 had between 6 to 10 years, and 3 had more than 11 years of experience. Participants used a language in which they stated proficiency (defined as language in which they were comfortable designing and implementing whole programs). Here, 19 of the 21 participants used Python, one used C++, and the final participant used JavaScript.

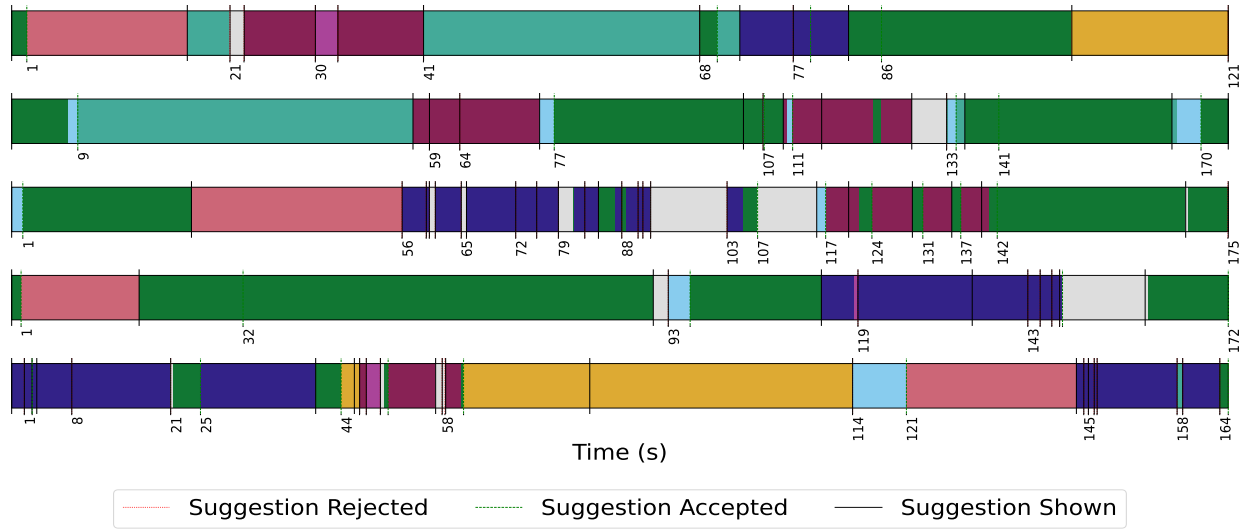
On average, participants took 12.23 minutes (sample standard deviation, $s_N = 3.98$ minutes) to complete the coding task, with a maximum session length of 20.80 minutes. This task completion time is measured from the first line of code written for the task until the end of the allocated time. During the coding tasks, Copilot showed participants a total of 1024 suggestions, out of which they accepted 34.0%. The average acceptance rate for participants was 36.5% (averaging over the acceptance rate of each participant), and the median was 33.8% with a standard error of 11.9%; the minimum acceptance rate was 14.3%, and the maximum was 60.7%. In the labeling phase, each participant labeled an average of 149.38 ($s_N = 57.43$) segments with CUPS, resulting in a total of 3137 CUPS labels. The participants used the ‘custom state’ text field only three times total, twice a participant wrote ‘write a few letters and expect suggestion’ which can be considered as ‘prompt crafting’ and once a participant wrote ‘I was expecting the function skeleton to show up[.]’ which was mapped to ‘waiting for suggestion’. The IDK button was used a total of 353 times, this sums to 3137 CUPS + 353 IDKs = 3490 labels, the majority of its use was from two participants (244 times) where the video recording was not clear enough during consecutive spans, and was used by only five other participants more than once with the majority of the use also being due to the video not being clear or the segment being too short. The IDK segments represent 6.5% of total session time across all participants, mostly contributed by five participants. Therefore, we remove the IDK segments from the analysis and do not attempt to re-label them.

Together, these CUPS labels enable us to investigate various questions about programmer-CodeRec interaction systematically, such as exploring which activities programmers perform most frequently and how they spend most of their time. We study programmer-CodeRec interaction using the data derived from this study in the following Section 6 and derive various insights and interventions.

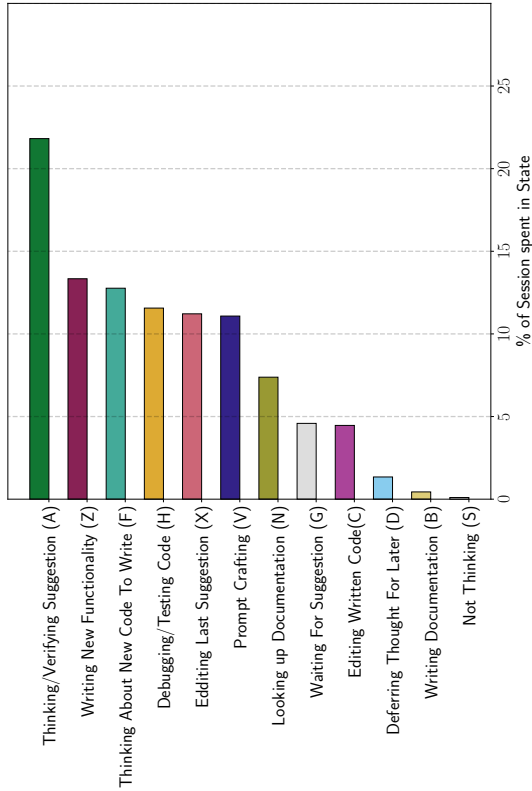
6 UNDERSTANDING PROGRAMMER BEHAVIOR WITH CUPS: MAIN RESULTS

The study in the previous section allows us to collect telemetry with CUPS labels for each telemetry segment. We now analyze the collected data and highlight suggestions for 1) **metrics** to measure the programmer-CodeRec interaction, 2) **design improvements** for the Copilot interface, and finally 3) **insights** into programmer behavior. Each subsection below presents a specific result or analysis which can be read independently. Code and Data is available at ⁵.

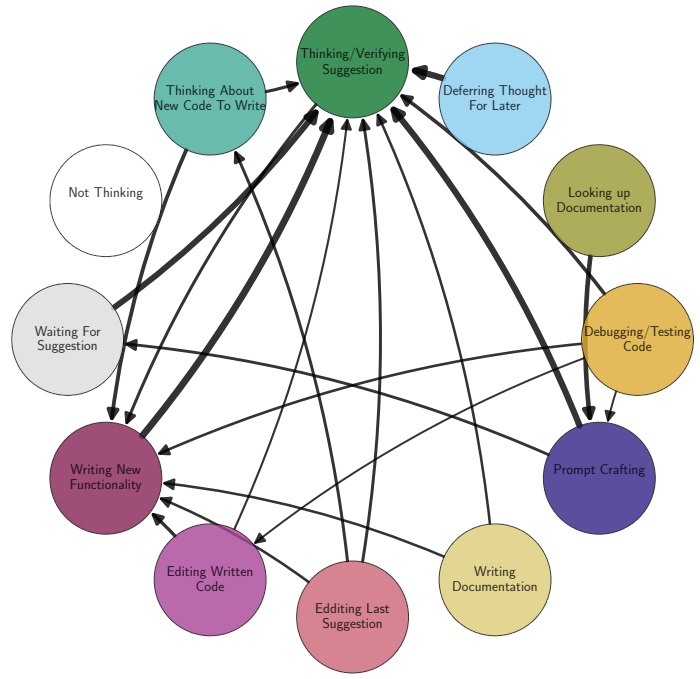
⁵https://github.com/microsoft/coderec_programming_states



(a) Individual CUPS timelines for 5/21 study participants for the first 180 secs show the richness of and variance in programmer-CodeRec interaction.



(b) The percentage of total session time spent in each state during a coding session. On average, verifying Copilot suggestions occupies a large portion of session time.



(c) CUPS diagram showing 12 CUPS states (nodes) and the transitions among the states (arcs). Transitions occur when a suggestion is shown, accepted, or rejected. We hide self-transitions and low-probability transitions for simplicity

Figure 5: Visualization of CUPS labels from our study as timelines, a histogram, and a state machine.

Table 2: Description of the coding tasks given to user study participants and task assignment. Participants were randomly allocated to tasks for tasks which they had familiarity with.

Task Name	Participants	Description
Algorithmic Problem	P4,P17,P18	Implementation of TwoSum, ThreeSum and FourSum
Data Manipulation	P1,P2,P11,P20	Imputing data with average feature value and feature engineering for quadratic terms
Data Analysis	P5,P8	Computing data correlations in a matrix and plotting of most highly correlated features
Machine Learning	P3,P7,P12,P15	Training and Evaluation of models using sklearn on given dataset
Classes and Boilerplate Code	P6,P9	Creating different classes that build on each other
Writing Tests	P16	Writing tests for a black box function that checks if a string has valid formatting
Editing Code	P10,P14,P21	Adding functionality to an existing class that implements a nearest neighbor retriever
Logistic Regression	P13,P19	Implementing a custom Logistic Regression from scratch with weight regularization

6.1 Aggregated Time Spent in Various CUPSs

In Figure 5a, we visualize the coding sessions of *individual* participants as *CUPS timelines*, where each telemetry segment is labeled with its CUPS label. At first glance, CUPS timelines show the richness in patterns of interaction with Copilot, as well as the variance in usage patterns across settings and people. CUPS timelines allow us to inspect individual behaviors and identify patterns, which we later aggregate to form general insights into user behavior.

Figure 5b shows the average time spent in each state as a percentage normalized to a user’s session duration.

Metric Suggestion: Time spent in CUPS states as a high-level diagnosis of the interaction

For example, time spent ‘Waiting For Suggestion’ (4.2%, $s_N = 4.46$) measures the real impact of **latency**, and time spent ‘Editing Last Suggestion’ provides feedback on the quality of suggestions.

We find that averaged across all users, the **‘verifying suggestion’ state takes up the most time** at 22.4% ($s_N = 12.97$), it is the top state for 6 participants and in the top 3 states for 14 out of 21 participants taking up at least 10% of session time for all but one participant. Notably, this is a new programmer task introduced by Copilot. The second-lengthiest state is writing new functionality’ 14.05% ($s_N = 8.36$), all but 6 participants spend more than 9% of session time in this state.

More generally, the states that are specific to interaction with Copilot include: ‘Verifying Suggestions’, ‘Deferring Thought for Later’, ‘Waiting for Suggestion’, ‘Prompt Crafting’, and ‘Editing Suggestion’. **We found that the total time participants spend in these states is 51.5 % ($s_N = 19.3$) of the average session duration.** In fact, half of the participants spend more than 47% of their session in these Copilot states, and all participants spend more than 21% of their time in these states.

By Programmer Expertise and Copilot Experience. We investigate if there are any differences in how programmers interacted with Copilot based on their programming expertise and their previous experience with Copilot. First, we split participants based on whether they have professional programming experience of more

than 6 years (10 out of 21) and who have less than 6 years (11 out of 21). We notice the acceptance rate for those with substantial programming experience is $30.0\% \pm 14.5$ while for those without is $37.6\% \pm 14.6$. Second, we split participants based on whether they had used Copilot previously (10 out of 21) and those who had never used it before (11 out of 21). The acceptance rate for those who have previously used Copilot is $37.6\% \pm 15.3$, and for those who have not, it is 29.3 ± 13.7 . Due to the limited number of participants, these results are not sufficient to determine the influence of programmer experience or Copilot experience on behavior. We also include in Appendix a breakdown of programmer behavior by task solved.

6.2 Patterns in Behavior as Transitions Between CUPS States

To understand if there was a pattern in participant behavior, we modeled *transitions* between two states as a *state machine*. We refer to the state machine-based model of programmer behavior as a *CUPS diagram*. In contrast to the timelines in Figure 5a, which visualize state transitions with changes of colors, the CUPS diagram Figure 5c explicitly visualizes transitions using directed edges, where the thickness of arrows is proportional to the likelihood of transition. For simplicity, Figure 5c only shows transitions with an average probability higher than 0.17 (90th quantile, selected for graph visibility).

The transitions in Figure 5c revealed many expected patterns. For example, one of the most likely transitions (excluding self-transitions from the diagram), ‘Prompt Crafting $\xrightarrow{0.54}$ Verifying Suggestion’ showed that when programmers were engineering prompts, they were then likely to immediately transition to verifying the resultant suggestions (probability of 0.54). Likewise, Another probable transition was ‘Deferring Thought $\xrightarrow{0.54}$ Verifying Suggestion’, indicating that **if a programmer previously deferred their thought for an accepted suggestion, they would, with high probability, return to verify that suggestion.** Stated differently: deference incurs verification debt, and this debt often “catches up” with the programmer. Finally, the single-most probable transition, ‘Writing New Functionality $\xrightarrow{0.59}$ Verifying Suggestion’, echos the observation from the previous section, indicating that programmers often see suggestions while writing code (rather than prompt crafting), then spend time verifying it. If suggestions are unhelpful, they could easily be seen as interrupting the flow of writing.

The CUPS diagram also revealed some unexpected transitions. Notably, the second-most probable transition from the 'Prompt Crafting' state is 'Prompt Crafting $\xrightarrow{0.25}$ Waiting for Suggestion'. This potentially reveals an unexpected and unnecessary delay and is a possible target for refinement (e.g., by reducing latency in Copilot). Importantly, each of these transitions occurs with a probability that is much higher than the lower bound/uniform baseline probability of transitioning to a random state in the CUPS diagram ($1/12=0.083$). In fact, when we compute the entropy rate (a measure of randomness) of the resulting Markov Chain [10] from the CUPS diagram we obtain a rate of 2.24; if the transitions were completely random then the rate would be 3.58, and if the transitions were deterministic then the rate is 0.

Interface Design Suggestion: Identifying current CUPS state can help serve programmer needs

If we are able to know the current programmer CUPS state during a coding session we can better serve the programmer, for example,

- If the programmer is observed to have been deferring their thought on the last few suggestions, group successive Copilot suggestions and display them together.
- If the programmer is waiting for the suggestion, we can prioritize resources for them at that moment
- While a user is prompt crafting, Copilot suggestions are often ignored and may be distracting; however, after a user is done with their prompt, they may expect high-quality suggestions. We could suppress suggestions during prompt crafting, but after the prompt crafting process is done, display multiple suggestions to the user and encourage them to browse through them.

Future work can, for example, realize these design suggestions by allowing **custom keyboard macros** for the programmer to signal their current CUPS state, or a more automated approach by **predicting** their CUPS state.

We also investigated longer patterns in state transitions by searching for the most common sequence of states of varying lengths. We achieved this by searching over all possible segment n-grams and counting their occurrence over all sessions. We analyzed patterns in two ways: in Figure 6a, we merged consecutive segments that have the same state label into a single state (thus removing self-transitions), and in Figure 6b we looked at n-grams in the user timelines (including self-transitions) where we include both states and participants actions (shown, accepted and rejected). The most common pattern (#1) in Figure 6a was a cycle where programmers repeatedly wrote new code functionality and then spent time verifying shown suggestions, indicating a new mode for programmers to solve coding tasks. At the same time, when we look at pattern (#B) in Figure 6b, which takes a closer look into when programmers are writing new functionality, we observe that they don't stop

to verify suggestions and reject them as they continue to write. Other long patterns include (#2) (also shown as pattern #D), where programmers repeatedly accepted successive Copilot suggestions after verifying each of them. Finally, we observe in (#3) and (#A) programmers iterating on the prompt for Copilot until they obtain the suggestion they want. We elaborate more on this in the next subsection.

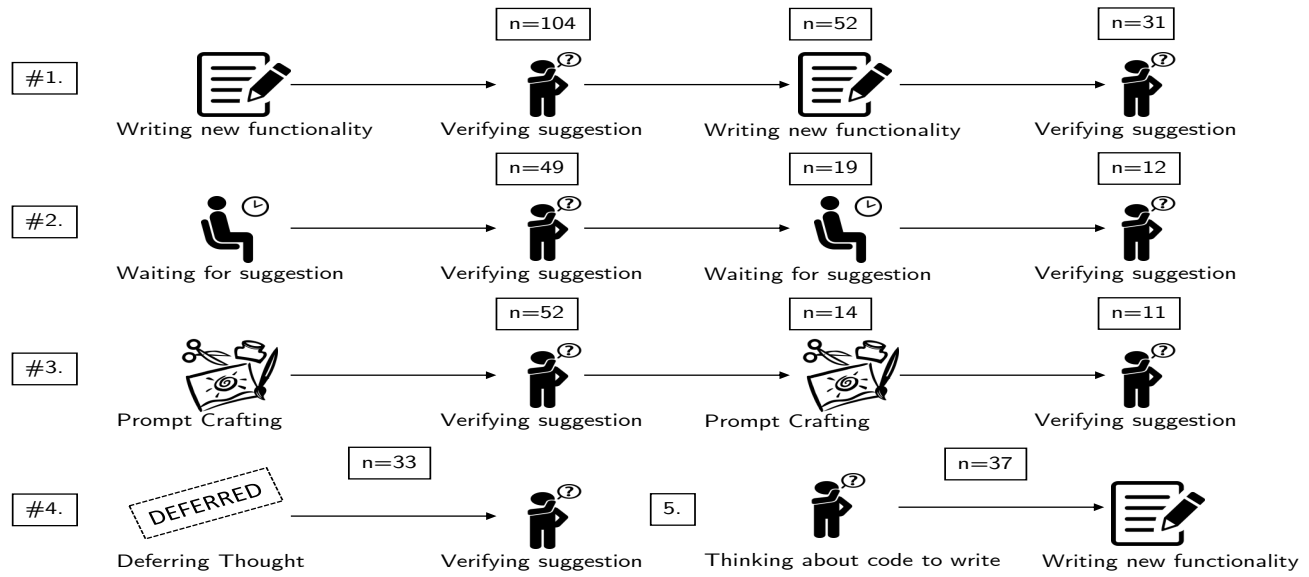
6.3 Programmers Often Defer Thought About Suggestions

An interesting CUPS state is that of 'Deferring Thought About A Suggestion'. This is illustrated in Figure 7, where programmers accept a suggestion or series of suggestions without sufficiently verifying them beforehand. This occurs either because programmers wish to see the suggestion with code highlighting, or because they want to see where Copilot suggestions leads to. Figure 5b shows that programmers do in fact, frequently defer thought— we counted 61 states labeled as such. What drives the programmer to defer their thought about a suggestion rather than immediately verifying it? We initially conjectured that the act of deferring may be partially explained by the length of the suggestions. So, we compared the number of characters and the number of lines for suggestions depending on the programmer's state. We find that there is no statistical difference according to a two-sample independent t-test ($t = -0.58, p = 0.56$)⁶ in the average number of characters between deferred thought and suggestions (75.81 compared to 69.06) that were verified previously. The same holds for the average number of lines.

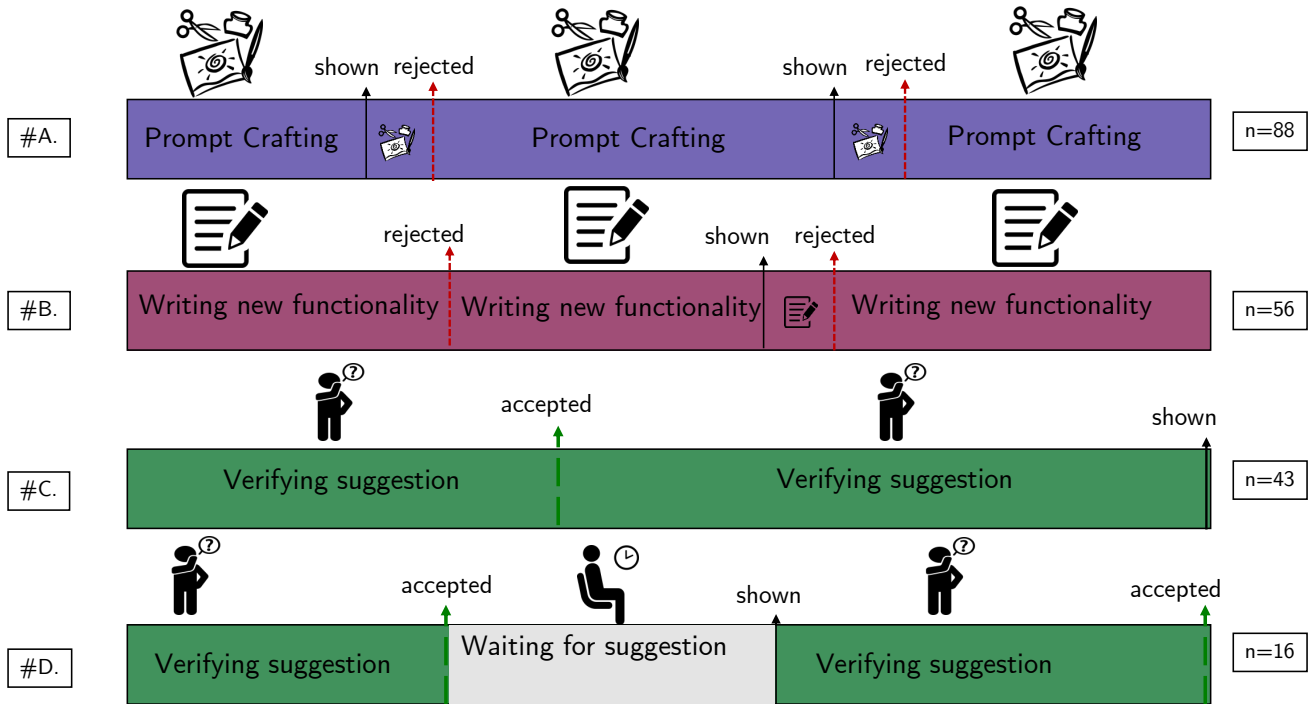
However, when we look at the likelihood of editing an accepted suggestion, we find that it is 0.18 if it was verified before, but it is 0.53 if it was deferred. This difference is significant according to a chi-square test ($\chi^2 = 29.2, p = 0$). In fact, the programmer CUPS state has a big effect on their future actions. In Table 3, we show the probability of the programmer accepting a suggestion given the CUPS state the programmer was in while the suggestion is being shown. We also show the probability of the programmer accepting a suggestion as a function of the CUPS state the programmer was in just before the suggestion was displayed. We observe there is a big variation in the suggestion acceptance rate by the CUPS state. For example, if the programmer was in the "Deferring Thought For Later" state, the probability of acceptance is 0.98 ± 0.02 compared to when a programmer is thinking about new code to write, where the probability is 0.12 ± 0.04 . Note that the average probability of accepting a suggestion was 0.34.

What are the programmers doing before they accept a suggestion? We found that the average probability of accepting a suggestion was 0.34. However, we observed that when the programmer was verifying a suggestion their likelihood of accepting was 0.70. In contrast, if the programmer was thinking about new code to write, the probability dropped to 0.20. This difference was statistically significant according to Pearson's chi-squared test ($\chi^2 = 12.25, p = 0$). Conversely, when programmers are engineering prompts, the likelihood of accepting a suggestion drops to 0.16. One reason for this might be that programmers want to write the prompt on their

⁶All p-values reported are corrected for multiple hypothesis testing with the Benjamin/Hochberg procedure with $\alpha = 0.05$.



(a) Common patterns of transitions between *distinct* states. In individual participant timelines, the patterns visually appear as a change of color, but here we measure how often they appear across all participants (n=).



(b) Common patterns of states and actions (including self transitions). Each pattern is extracted from user timelines and we count how often it appears in total (n=)

Figure 6: Myriad of CUPS patterns observed in our study.

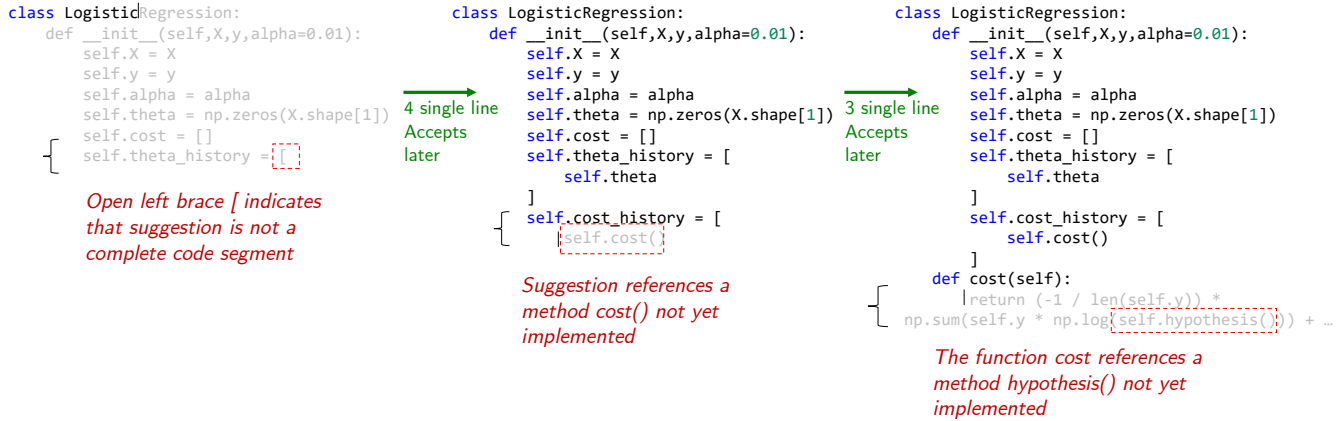


Figure 7: Illustration of a coding scenario with Copilot where the programmer may choose to defer verifying a suggestion (‘Deferring Thought’). Here, Copilot suggests an implementation for the class Logistic Regression line-by-line (illustrated from left to right). And the programmer may need to defer verifying intermediate suggestion of `self.cost` (middle screenshot) because the method that implemented it is suggested later (right screenshot).

own without suggestions, and Copilot interrupts them. We show the full results in the Appendix for the other states.

Table 3: We compute the percentage of suggestions accepted given the programmer was in the CUPS state while the suggestion is being shown (% Ss accepted while shown). We compute the percentage of suggestions accepted given the programmer was in the CUPS state before the suggestion is shown, the state just before the one where the suggestion is shown (% Ss accepted before S is shown). We compute the standard error for the acceptance rate (%).

State	% Ss accepted while shown	% Ss accepted before S is shown
Thinking/Verifying Suggestion	0.80 ± 0.02	0.56 ± 0.04
Prompt Crafting	0.11 ± 0.02	0.22 ± 0.03
Looking up Documentation	0.00 ± 0.00	0.29 ± 0.17
Writing New Functionality	0.07 ± 0.02	0.31 ± 0.03
Thinking About New Code To Write	0.12 ± 0.04	0.27 ± 0.04
Editing Last Suggestion	0.03 ± 0.03	0.23 ± 0.05
Waiting For Suggestion	0.10 ± 0.05	0.58 ± 0.06
Editing Written Code	0.07 ± 0.04	0.17 ± 0.07
Writing Documentation	0.40 ± 0.22	0.33 ± 0.19
Debugging/Testing Code	0.23 ± 0.07	0.26 ± 0.06
Deferring Thought For Later	0.98 ± 0.02	1.0 ± 0.0

6.4 CUPS Attributes Significantly More Time Verifying Suggestions than Simpler Metrics

We observed that programmers continued verifying the suggestions after they accepted them. This happens by definition for ‘deferred thought’ states before accepting suggestions, but we find it also happens when programmers verify the suggestion before accepting it and this leads to a significant increase in the total time verifying suggestions. First, when participants defer their thought about a suggestion they accepted, 53.2% of the time they verify the suggestion immediately afterward. When we adjust for the post-hoc time spent verifying, we compute a mean time of 15.21 ($s_N = 20.68$) seconds of verification and a median time of 6.48s. This is nearly a five-times increase in average time and a three-time increase in median time for the pre-adjustment scores of 3.25 ($s_N = 3.33$) mean and 1.99 median time. These results are illustrated in Figure 8 and is a statistically significant increase according to a two-sample paired t-test ($t = -4.88, p = 1.33 \cdot 10^{-5}$). This phenomenon also occurs when programmers are in a ‘Thinking/Verifying Suggestion’ state before accepting a suggestion where 19% of the time they posthoc verify the suggestion which increases total verification time from 3.96 ($s_N = 8.63$) to 7.03 ($s_N = 14.43$) on average which is statistically significant ($t = -4.17, p = 5e - 5$). On the other hand, programmers often have to wait for suggestions to show up due to either latency or Copilot not kicking in to provide a suggestion. If we sum the time between when a suggestion is shown and the programmer accepts or rejects this in addition to the time they spend waiting for the suggestion (this is indicated in the state ‘Waiting for suggestion’), then we get an increase from 6.11s ($s_N = 15.52$) to 6.51s ($s_N = 15.61$) which is minor on average but adds 2.5 seconds of delay when programmers have to explicitly wait for suggestions.

Metric Suggestion: Adjust verification time metrics and acceptance rates to include suggestions that are verified after acceptance

The previous analysis showed that the time to accept a suggestion cannot be simply measured as the time spent from the instance a suggestion is shown until a suggestion is accepted—this misses the time programmers spend verifying a suggestion after acceptance. Similarly, since deferring thought is a frequent behavior observed, it leads to an inflation of acceptance rates. We recommend using measures such as the fraction of suggestions accepted that survive in the codebase after a certain time period (e.g. 10 minutes).

6.5 Insights About Prompt Crafting

Insights about Prompt Crafting. We take a closer look into how participants craft prompts to obtain Copilot suggestions. *Our first insight is that programmers consistently ignore suggestions while prompt crafting. Among 234 suggestions that were shown while participants were actively prompt crafting, defined as a suggestion where a programmer was prompt crafting while the suggestion was being displayed, only 10.7% were accepted.* We hypothesize this behavior could be due to programmers wanting to craft the prompt in their own language rather than relying on Copilot to help them prompt craft. This also indicates that Copilot is unnecessarily interrupting participants' prompt crafting attempts.

However, programmers often iterate on their prompts until they obtain the suggestion they desire and often do not abandon prompt crafting without accepting a suggestion. We define a prompt crafting attempt as a segment of the coding session that starts from when the programmer first enters the CUPS "prompt crafting" state and lasts until the programmer enters a non-Copilot centric state⁷. We count 59 such prompt crafting attempts wherein 81.3% of them a suggestion is accepted.

Prompt crafting is often an iterative process, where the programmer writes an initial prompt, observes the resulting suggestion, then iterates on the prompt by adding additional information about the desired code or by rewording the prompt. For example, P5 wanted to retrieve the index of the maximum element in a correlation matrix and wrote this initial prompt and got the suggestion:

```
# print the indices of the max value excluding 1 in corr
maxval = np.amax(corr, axis=1) # Copilot suggestion
```

This code snippet returns the value of the maximum value rather than the index, so it was not accepted by the participant. They then re-wrote the prompt to be:

```
# print the two features most correlated
# Copilot suggestion
maxcor = np.where(corr == np.amax(corr))
```

and accepted the above suggestion.

⁷The non-Copilot centric states are: 'Writing New Functionality,' 'Editing Written Code,' 'Thinking About New Code To Write,' 'Debugging/Testing Code,' 'Looking up Documentation,' 'Writing Documentation.'

Finally, we observe that there are three main ways participants craft prompts:

1) through writing a single line comment with natural language instructions, although the comment may resemble pseudo-code [17], an example:

```
# impute missing values in X_train as average of column
# where missing value is -1
```

2) through writing a docstring for the function:

```
def distance(self, query):
    ...
    query: single numpy array
    return: 12 distances from query to the vectors
    ...
```

and finally, 3) through writing function signatures (or variable names) e.g., writing "def add_time" then pausing to wait for a suggestion. Often, programmers combine the three prompt crafting strategies to get better code suggestions.

6.6 Post-Study Survey Answers

After completing the study, participants were asked to complete a survey based on the productivity survey in [40], which focuses on the SPACE framework of programmer productivity [13]. We also included a free-form text box at the end of the survey where participants can add any additional thoughts about their experience using Copilot for the task assigned. The full results of the survey can be found in the Appendix.

We found that 6/21 participants agreed or strongly agreed with the statement that they were concerned about the quality of their code when using Copilot. Participant #9 noted, "I worry that bugs can sneak-in and go unnoticed, especially in weakly-dynamically typed languages" and Participant #19 noted that "My main concern with Copilot is whether it is teaching me to do things the wrong (or old) way (e.g. showing me a Python 3.6 way instead of a Python 3.10 way and so on)". On the other hand, 14/21 participants agreed that using Copilot helped them stay in flow and spend less time searching for information. Participant #3 noted that "Collaborating with Copilot felt like I was googling what I wanted to do except instead of going through several stack overflow links that Google would show me, the code just appeared inline saving me time and keeping my flow of coding" and Participant #6 "Going into the exercise I genuinely thought there would be a point when I pull up stack overflow. Because that's the kind of tiny stuff you sometimes need to search for. With copilot, it really reduced my worry of doing so." Finally, 17/21 participants agreed with the statement that by using Copilot, they completed the task faster, and 16/21 participants agreed that they were more productive using Copilot. These survey responses highlight the costs and benefits of writing code with Copilot and reinforcing existing results in [40].

7 LIMITATIONS, FUTURE WORK AND CONCLUSION

7.1 Limitations

The observations from our study are limited by several decisions that we made. First, our participants solved time-limited coding

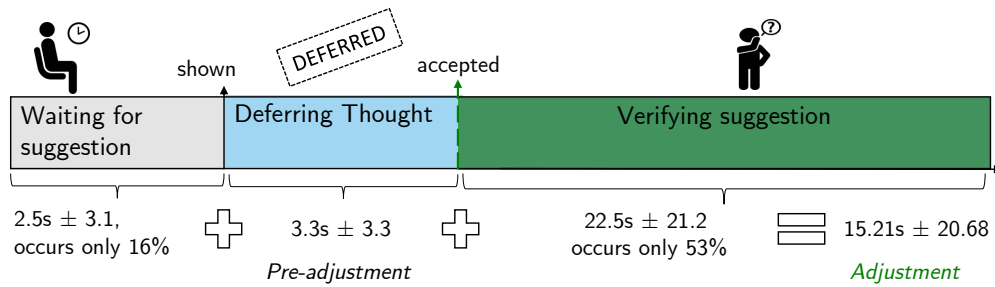


Figure 8: Illustration of one of the adjustments required for measuring the total time a programmer spends to verify a suggestion. Here, when a programmer defers thought for a suggestion, they spend time verifying it after accepting it and may also have to wait beforehand for the suggestion to be shown.

tasks that were provided by us instead of real tasks they may perform in the real world. Furthermore, the selection of tasks was limited and did not cover all tasks programmers might perform. We mostly conducted experiments with Python with only two participants using C++ and JavaScript when Copilot is capable of completing suggestions for myriads of other languages. We also made an assumption about the granularity of telemetry where each segment at most contained one state when, in a more general setting, programmers may perform multiple activities within a single segment. We also did not capture longer-term costs of interacting, e.g., from accepting code with security vulnerabilities or longer horizon costs. To this end, security vulnerabilities and possible overreliance issues [2, 25, 26], are important areas of research that we do not address in this paper.

7.2 Future Work

We only investigated a limited number of programmer behaviors using the CUPS timelines and diagrams. There are many other aspects future work could investigate.

Predicting CUPS states. To enable our insights derived in Section 6, we need to be able to identify the current programmer's CUPS state. An avenue towards that is building predictive models using labeled telemetry data that is collected from our user study. Ideally, we can leverage this labeled data to further label telemetry data from other coding sessions or other participants so that we can perform such analyses more broadly. Specifically, the input to such a model would be the current session context, for example, whether the programmer accepted the last suggestion, the current suggestion being surfaced, and the current prompt. We can leverage supervised learning methods to build such a model from collected data. Such models would need to run in real-time during programming and predict at each instance of time the current user CUPS state. This would enable the design suggestions proposed to serve to compute various metrics proposed. For example, if the model predicts that the programmer is deferring thought about a suggestion, we can group suggestions together to display them to the programmer. In the Appendix, we built small predictive models of programmers CUPS state using labeled study data. However, the current amount of labeled data is not sufficient to build highly accurate models. There are multiple avenues to improve the performance of these models: 1) simply collecting a larger amount

of labeled data which would be expensive, 2) using methods from semi-supervised learning that leverage unlabeled telemetry to increase sample efficiency [35], and 3) collecting data beyond what is captured from telemetry such as video footage of the programmer screen (e.g. cursor movement) to be able to better predict with the same amount of data.

Assessing Individual Differences. There is an opportunity to apply the CUPS diagram to compare different user groups and compare how individuals differ from an average user. Does the nature of inefficiencies differ between user groups? Can we personalize interventions? Finally, we could also compare how the CUPS diagram evolves over time for the same set of users.

Effect of Conditions and Tasks on Behavior. We only studied the behavior of programmers with the current version of Copilot. Future work could study how behavior differs with different versions of Copilot— especially when versions use different models. In the extreme, we could study behavior when Copilot is turned off. The latter could help assess the *counterfactual* cost of completing the task without AI assistance and help establish whether and where Copilot suggestions add net value for programmers. For example, maybe the system did not add enough value because the programmer kept getting into prompt crafting rabbit holes instead of moving on and completing the functions manually or with the assistance of web search.

Likewise, if developers create a faster version of Copilot with less latency, the CUPS diagram could be used to establish whether it leads to reductions in time spent in the "Waiting for Suggestion" state.

Informing New Metrics. Since programmers' value may be multi-dimensional, how can we go beyond code correctness and measure added value for users? If Copilot improves productivity, which aspects were improved? Conversely, if it didn't, where are the inefficiencies? One option is to conduct a new study where we compare the CUPS diagram with Copilot assistance with a counterfactual condition where the programmers don't have access to Copilot. And use the two diagrams to determine where the system adds value or could have added value. For example, the analysis might reveal that some code snippets are too hard for programmers to complete by themselves but much faster with Copilot because the cost of double-checking and editing the suggestion is much less

than the cost of spending effort on it by themselves. Conversely, the analysis might reveal that a new intervention for helping engineer prompts greatly reduced people's times in "Prompt Crafting".

Another option is to design offline metrics based on these insights that developers can use during the model selection and training phase. For example, given that programmers spent a large fraction of the time verifying suggestions, offline metrics that can estimate this (e.g., based on code length and complexity) may be useful indicators of which models developers should select for deployment. Future work will aim to test the effectiveness of these design suggestions as well.

Beyond Programming. We also hope our methodology is applied to study other forms of AI assistants that are rapidly being deployed. For example, one can make an analogous CUPS taxonomy for writing assistants for creative writers or lawyers.

7.3 Conclusion

We developed and proposed a taxonomy of common programmer activities (CUPS) and combined it with real-time telemetry data to profile the interaction. At present, CUPS contains 12 mutually unique activities that programmers perform between consecutive Copilot actions (e.g., such as accepting, rejecting, and viewing suggestions). We gathered real-world instance data of CUPS by conducting a user study with 21 programmers within our organization, where they solved coding tasks with Copilot and retrospectively labeled CUPS for their coding session. We collected over 3137 instances of CUPS and analyzed them to generate CUPS timelines that show individual behavior and CUPS diagrams that show aggregate insights into the behavior of our participants. We also studied the time spent in these states, patterns in user behavior, and better estimates of the cost (in terms of time) of interacting with Copilot.

Our studies with CUPS labels revealed that when solving a coding task with Copilot, programmers may spend a large fraction of total session time (34.3%) on just double-checking and editing Copilot suggestions, and spend more than half of the task time on Copilot related activities, together indicating that introducing Copilot into an IDE can significantly change user behavior. We proposed new metrics to measure the interaction by computing the time spent in each CUPS state and modification to existing time and acceptance metrics by accounting for suggestions that get verified only after they get accepted. We proposed a new interface design suggestion: if we allow programmers to signal their current state, then we can better serve their needs, for example, by reducing latency if they are waiting for a suggestion.

ACKNOWLEDGMENTS

HM partly conducted this work during an internship at Microsoft Research (MSR). We acknowledge valuable feedback from colleagues across MSR and GitHub including Saleema Amershi, Victor Dibia, Forough Poursabzi, Andrew Rice, Eirini Kalliamvakou, and Edward Aftandilian.

REFERENCES

- [1] Amazon. 2022. ML-powered coding companion – Amazon CodeWhisperer. <https://aws.amazon.com/codewhisperer/>

- [2] Owura Asare, Meiyappan Nagappan, and N Asokan. 2023. Is github's copilot as bad as humans at introducing vulnerabilities in code? *Empirical Software Engineering* 28, 6 (2023), 1–24.
- [3] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [4] Ruven Brooks. 1977. Towards a theory of the cognitive processes in computer programming. *International Journal of Man-Machine Studies* 9, 6 (1977), 737–751.
- [5] Ruven E Brooks. 1980. Studying programmer behavior experimentally: The problems of proper methodology. *Commun. ACM* 23, 4 (1980), 207–213.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [7] Stuart K Card, Thomas P Moran, and Allen Newell. 1980. The keystroke-level model for user performance time with interactive systems. *Commun. ACM* 23, 7 (1980), 396–410.
- [8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* 1, 1 (2021), 1–2.
- [9] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Zhen Ming Jack Jiang. 2023. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software* 203 (2023), 111734.
- [10] Laura Ekroot and Thomas M Cover. 1993. The entropy of Markov trajectories. *IEEE Transactions on Information Theory* 39, 4 (1993), 1418–1421.
- [11] Mikhail Evtikhiev, Egor Bogomolov, Yaroslav Sokolov, and Timofey Bryksin. 2023. Out of the bleu: how should we assess quality of the code generation models? *Journal of Systems and Software* 203 (2023), 111741.
- [12] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of developer productivity. *Commun. ACM* 64, 6 (2021), 46–53.
- [13] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. 2021. The SPACE of Developer Productivity: There's more to it than you think. *Queue* 19, 1 (2021), 20–48.
- [14] Github. 2022. GitHub copilot - your AI pair programmer. <https://github.com/features/copilot>
- [15] Vincent J Hellendoorn, Sebastian Proksch, Harald C Gall, and Alberto Bacchelli. 2019. When code completion fails: A case study on real-world completions. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, .., 960–970.
- [16] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. 2021. Measuring Coding Challenge Competence With APPS. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. .., 1–2.
- [17] Ellen Jiang, Edwin Toh, Alejandra Molina, Kristen Olson, Claire Kayacik, Aaron Donsbach, Carrie J Cai, and Michael Terry. 2022. Discovering the Syntax and Strategies of Natural Language Programming with Generative Language Models. In *CHI Conference on Human Factors in Computing Systems*. .., 1–19.
- [18] Bonnie E John and David E Kieras. 1996. The GOMS family of user interface analysis techniques: Comparison and contrast. *ACM Transactions on Computer-Human Interaction (TOCHI)* 3, 4 (1996), 320–351.
- [19] An Ju and Armando Fox. 2018. TEAMSCOPE: measuring software engineering processes with teamwork telemetry. In *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*. .., 123–128.
- [20] Eirini Kalliamvakou. 2022. Research: Quantifying github copilot's impact on developer productivity and happiness. <https://github.blog/2022-09-07-research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/>
- [21] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *arXiv preprint arXiv:2203.07814* 1, 1 (2022), 1–2.
- [22] Jenny T Liang, Chenyang Yang, and Brad A Myers. 2023. Understanding the Usability of AI Programming Assistants. *arXiv preprint arXiv:2303.17125* 1, 1 (2023), 1–2.
- [23] Henry Lieberman and Christopher Fry. 1995. Bridging the gulf between code and behavior in programming. In *Proceedings of the SIGCHI conference on Human factors in computing systems*. .., 480–486.
- [24] Unaizah Obaidallah, Mohammed Al Haek, and Peter C-H Cheng. 2018. A survey on the usage of eye-tracking in computer programming. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–58.
- [25] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the keyboard? assessing the security of github copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy*

- (SP). IEEE, , , 754–768.
- [26] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2021. Can OpenAI Codex and Other Large Language Models Help Us Fix Security Bugs? *arXiv preprint arXiv:2112.02125* 1, 1 (2021), 1–2.
 - [27] Norman Peitek, Janet Siegmund, and Sven Apel. 2020. What drives the reading order of programmers? an eye tracking study. In *Proceedings of the 28th International Conference on Program Comprehension*. , , 342–353.
 - [28] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* 1, 1 (2023), 1–2.
 - [29] James Prather, Brent N Reeves, Paul Denny, Brett A Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. "It's Weird That it Knows What I Want": Usability and Interactions with Copilot for Novice Programmers. *arXiv preprint arXiv:2304.02491* 1, 1 (2023), 1–2.
 - [30] Advait Sarkar, Andrew D Gordon, Carina Negreanu, Christian Poelitz, Sruti Srinivasa Ragavan, and Ben Zorn. 2022. What is it like to program with artificial intelligence? *arXiv preprint arXiv:2208.06213* 1, 1 (2022), 1–2.
 - [31] Beau A Sheil. 1981. The psychological study of programming. *ACM Computing Surveys (CSUR)* 13, 1 (1981), 101–120.
 - [32] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, Mingze Ni, and Li Li. 2023. Don't Complete It! Preventing Unhelpful Code Completion for Productive and Sustainable Neural Code Completion Systems. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, , , 324–325.
 - [33] Maxim Tabachnyk Tabachnyk and Stoyan Nikolov. 2022. ML-enhanced code completion improves developer productivity. <https://ai.googleblog.com/2022/07/ml-enhanced-code-completion-improves>
 - [34] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. , , 1–7.
 - [35] Jesper E Van Engelen and Holger H Hoos. 2020. A survey on semi-supervised learning. *Machine learning* 109, 2 (2020), 373–440.
 - [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017), 1–2.
 - [37] Zdenek Velart and Petr Šaloun. 2006. User behavior patterns in the course of programming in C++. In *Proceedings of the joint international workshop on Adaptivity, personalization & the semantic web*. , , 41–44.
 - [38] Justin D Weisz, Michael Muller, Stephanie Houde, John Richards, Steven I Ross, Fernando Martinez, Mayank Agarwal, and Kartik Talamadupula. 2021. Perfection not required? Human-AI partnerships in code translation. In *26th International Conference on Intelligent User Interfaces*. IUI, IUI, 402–412.
 - [39] Tongshuang Wu, Kenneth Koedinger, et al. 2023. Is AI the better programming partner? Human-Human Pair Programming vs. Human-AI pAIr Programming. *arXiv preprint arXiv:2306.05153* 1, 1 (2023), 1–2.
 - [40] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. ACM, ACM, 21–29.